

Day 20: タスク検索機能を実装しよう

今日のゴール

キーワードでタスクを検索できる機能を実装します。タイトルと説明文から該当するタスクを見つけられるようにします。

【スクリーンショット: 検索画面と結果】

なぜこれを作るのか?

タスクが増えると、目的のタスクを探すのが大変になります。**検索機能は図書館の検索端末のようなもの**。本棚を端から端まで探さなくても、キーワードを入力すれば目的の本がすぐ見つかります。同様に、検索機能があれば、大量のタスクの中から必要なものをすぐに見つけられます。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	検索ページ作成	10分
Step 2	検索フォーム実装	15分
Step 3	検索API呼び出し	15分
Step 4	検索結果の表示	15分

合計時間: 約55分

Step 1: 検索ページ作成 (10分)

実装:

```
// filepath: src/app/search/page.tsx
'use client';

import { Box, Typography } from '@mui/material';

export default function SearchPage() {
  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4">タスク検索</Typography>
    </Box>
  );
}
```

確認ポイント: /searchにアクセスして「タスク検索」が表示される

Step 2: 検索フォーム実装 (15分)

実装:

```
// filepath: src/app/search/page.tsx
import { useState } from 'react';
import { TextField, Button, InputAdornment } from '@mui/material';
import SearchIcon from '@mui/icons-material/Search';

export default function SearchPage() {
  const [query, setQuery] = useState('');

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    console.log('検索:', query);
  };

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>タスク検索</Typography>

      <form onSubmit={handleSearch}>
        <Box sx={{ display: 'flex', gap: 2, maxWidth: 600 }}>
          <TextField
            fullWidth
            placeholder="タスクを検索..."
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            InputProps={{
              startAdornment: (
                <InputAdornment position="start">
                  <SearchIcon />
                </InputAdornment>
              ),
            }}
          />
          <Button
            type="submit"
            variant="contained"
            disabled={!query.trim()}
          />
        </Box>
      </form>
    </Box>
  );
}
```

```
        >  
        検索  
    </Button>  
  </Box>  
</form>  
</Box>  
);  
}
```

確認ポイント: 検索フォームが表示され、入力できる

Step 3: 検索API呼び出し（15分）

実装:

```

// filepath: src/app/search/page.tsx
import { api } from '@trpc/react';
import { CircularProgress } from '@mui/material';

export default function SearchPage() {
  const [query, setQuery] = useState('');
  const [searchQuery, setSearchQuery] = useState('');

  const { data: results, isLoading } = api.search.tasks.useQuery(
    { query: searchQuery },
    { enabled: searchQuery.length > 0 }
  );

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    if (query.trim()) {
      setSearchQuery(query.trim());
    }
  };

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>タスク検索</Typography>

      <form onSubmit={handleSearch}>
        { /* フォーム内容は同じ */ }
      </form>

      {isLoading && (
        <Box sx={{ display: 'flex', justifyContent: 'center' }}>
          <CircularProgress />
        </Box>
      )}

      {results && (
        <Typography variant="body2" color="text.secondary">

```

```
        {results.length}件の結果  
      </Typography>  
    )}  
  </Box>  
);  
}
```

確認ポイント: 検索ボタンをクリックすると件数が表示される

Step 4: 検索結果の表示（15分）

実装:

```
// filepath: src/app/search/page.tsx
import {
  Card,
  CardContent,
  CardActionArea,
  Chip,
  Box,
} from '@mui/material';
import Link from 'next/link';

export default function SearchPage() {
  return (
    <Box sx={{ p: 3 }}>
      {/* フォーム部分 */}

      {results && results.length > 0 && (
        <Box sx={{ mt: 3 }}>
          {results.map((task) => (
            <Card key={task.id} sx={{ mb: 2 }}>
              <CardActionArea
                component={Link}
                href={`\`/projects/${task.projectId}/tasks\`}
              >
                <CardContent>
                  <Box sx={{ display: 'flex', gap: 1, mb: 1 }}>
                    <Chip
                      label={task.project.name}
                      size="small"
                      sx={{
                        bgcolor: task.project.color,
                        color: 'white',
                      }}
                    />
                    <Chip label={task.status} size="small" />
                    <Chip label={task.priority} size="small" />
                  </Box>
                </CardContent>
              </CardActionArea>
            </Card>
          ))}
        </Box>
      )}
    </Box>
  );
}
```



```

        <Typography variant="h6">{task.title}</
        {task.description && (
          <Typography
            variant="body2"
            color="text.secondary"
            sx={{
              overflow: 'hidden',
              textOverflow: 'ellipsis',
              display: '-webkit-box',
              WebkitLineClamp: 2,
              WebkitBoxOrient: 'vertical',
            }}
          >
            {task.description}
          </Typography>
        )}
      </CardContent>
    </CardActionArea>
  </Card>
)}
</Box>
)}

{results && results.length === 0 && (
  <Typography variant="body1" sx={{ mt: 3, textAlig
    検索結果が見つかりませんでした
  </Typography>
)}
</Box>
);
}

```

確認ポイント: 検索結果がカード形式で表示される

学んだこと

- **InputAdornment**: テキストフィールドにアイコンを追加
- **enabled オプション**: 条件が満たされたときだけクエリを実行
- **2つのstate**: query (入力中) と searchQuery (確定後) を分離
- **WebkitLineClamp**: 複数行テキストの省略表示
- **textOverflow: 'ellipsis'**: 長いテキストを「...」で省略

今日のまとめ

- ☐ 検索ページを作成できた
- ☐ 検索フォームを実装できた
- ☐ 検索APIを呼び出せた
- ☐ 検索結果を表示できた

⚠ つまづきポイント

問題	原因	解決策
文字入力のたびにAPIが呼ばれる	onChange で直接 searchQuery を更新	submit 時だけ searchQuery を更新
結果が0件でもメッセージが出ない	results が undefined のケースを考慮していない	results && results.length === 0 で判定
説明文が長すぎる	省略処理がない	WebkitLineClamp で行数制限

次回予告

Day 21では、ダッシュボードに統計カードを表示します。